

Machine Learning SDLC - Step-by-Step Architecture & Code Mapping

This report breaks down the Machine Learning Software Development Life Cycle (ML SDLC / AI Engineering Lifecycle) implemented in the Face Morph Studio repository. Every phase of our computer vision pipeline corresponds precisely to industry-standard ML engineering practices: data ingestion, feature/landmark extraction, algorithmic modeling, batch verification, and interactive production deployment.

Phase 1: Data Acquisition & Ingestion (Data Engineering)

In the initial SDLC stage, raw domain data must be sourced, filtered, and standardized. In our project, we fetch pre-aligned face images from Labeled Faces in the Wild (LFW) using scikit-learn. We specifically filter the dataset for 'different person' pairs (target == 0), convert the RGB float arrays [0, 1] to standard BGR uint8 [0, 255] images, and persist deterministic pairs to disk using a seeded random generator (seed=42) to ensure strict experimental reproducibility.

src/fetch_data.py - Data sourcing, filtering, and normalization:

```
def main():
    lfw = fetch_lfw_pairs(subset='train', color=True, resize=1.0, funneled=True)
    pairs, targets = lfw.pairs, lfw.target
    diff_idx = np.where(targets == 0)[0] # Filter for different-identity pairs only
    rng = np.random.default_rng(42)     # Seeded RNG for reproducibility
    chosen = rng.choice(diff_idx, size=min(60, len(diff_idx)), replace=False)
    for rank, idx in enumerate(chosen):
        imgA = (pairs[idx, 0] * 255).astype(np.uint8)
        imgB = (pairs[idx, 1] * 255).astype(np.uint8)
        imwrite(os.path.join(out_dir, f'pair{rank:03d}_A.png'), cv2.cvtColor(imgA, cv2.COLOR_RGB2BGR))
```

Phase 2: Data Preprocessing & Feature Extraction (Keypoint Engineering)

Before geometric modeling can occur, raw pixel arrays must be preprocessed and mapped to structured feature representations. In face morphing, feature extraction corresponds to tracking dense 3D facial mesh keypoints. To make MediaPipe robust on small LFW crops (125x94 px), we apply 3x bicubic upscaling. Crucially, because MediaPipe only tracks the face mask, we engineer 8 synthetic boundary points (image corners + edge midpoints) and append them to the landmark array so the background participates continuously in the warp.

src/landmarks.py - Upscaling, mesh detection, and boundary feature appending:

```
def get_landmarks(img_bgr, upscale=3):
    # 1. Upscale for reliable MediaPipe inference on small face crops:
    img_bgr = cv2.resize(img_bgr, None, fx=upscale, fy=upscale, interpolation=cv2.INTER_CUBIC)
    result = _landmarker.detect(mp.Image(image_format=mp.ImageFormat.SRGB, data=cv2.cvtColor(...)))
    if not result.face_landmarks: return None, None
    # 2. Extract 478 3D facial mesh points (scaled to upscaled pixel dims):
    pts = np.array([[lm.x * w, lm.y * h] for lm in result.face_landmarks[0]], dtype=np.float64)
    # 3. Feature engineering: append 8 frame boundary anchor points:
    pts = np.vstack([pts, _boundary_points(w, h)])
    return pts, img_bgr
```

Phase 3: Model Architecture & Algorithmic Core (Geometric & Photometric Engine)

The core transformation algorithm combines topological graph generation and piecewise-affine warping. First, we compute the target shape at blend factor α via convex combination: $\text{pts_morph} = (1-\alpha)\text{ptsA} + \alpha\text{ptsB}$. Next, we construct a Delaunay triangulation graph (`cv2.Subdiv2D`) on this averaged shape. For each triangle patch, we derive the affine transform matrix (`cv2.getAffineTransform`), warp the pixel patch (`cv2.warpAffine`), and composite it using a filled polygon mask. Finally, warped frames are linearly cross-dissolved.

src/morph.py - Delaunay graph topology & piecewise-affine cross-dissolving:

```
def morph_images(img1, pts1, img2, pts2, alpha, triangles=None):
    # 1. Averaged target shape where topological triangulation is established:
    pts_morph = (1 - alpha) * pts1 + alpha * pts2
    if triangles is None:
        triangles = delaunay_triangulation(pts_morph, (w, h))
    # 2. Piecewise-affine warping of each triangle index triplet (i, j, k):
    for (i, j, k) in triangles:
        _warp_triangle(img1f, warped1, [pts1[i], pts1[j], pts1[k]], [pts_morph[i], ...])
        _warp_triangle(img2f, warped2, [pts2[i], pts2[j], pts2[k]], [pts_morph[i], ...])
    # 3. Photometric cross-dissolving across both warped source images:
    morphed = (1 - alpha) * warped1 + alpha * warped2
    return np.clip(morphed, 0, 255).astype(np.uint8), triangles
```

Phase 4: Batch Verification, QA Auditing & Artifact Pipeline (Testing)

In production ML engineering, individual algorithms must be validated in batch mode across extensive datasets. Our batch pipeline iterates over all candidate pairs, safely catching detection anomalies (skipping pairs where face mesh tracking fails without crashing), writing standardized attack morphs ($\alpha=0.5$), building 5-step interpolation strips ($\alpha = 0, 0.25, 0.5, 0.75, 1.0$), and logging QA metrics to a JSON audit manifest.

src/generate_morphs.py - Batch verification, exception handling, and manifest logging:

```
# Batch pipeline across N pairs with automated anomaly skipping & audit logging:
for k, pid in enumerate(pair_ids):
    ptsA, upA = get_landmarks(imgA); ptsB, upB = get_landmarks(imgB)
    if ptsA is None or ptsB is None: n_fail += 1; continue # QA exception handling
    morphed, triangles = morph_images(upA, ptsA, upB, ptsB, alpha=0.5)
    imwrite(os.path.join(OUT_MORPHS, f'{pid}_morph.png'), morphed)
    manifest.append(pid); n_ok += 1
with open(os.path.join(BASE, 'outputs', 'morph_manifest.json'), 'w') as f:
    json.dump(manifest, f, indent=2)
```

Phase 5: Production Deployment & Serving (Interactive Front-End)

The final stage of the ML SDLC exposes the core algorithmic engine through a live interactive interface. Built with Streamlit, our serving tier handles real-time user uploads and slider events. To ensure low latency during interactive alpha scrubbing, heavy feature extraction is wrapped in `@st.cache_data` decorators. Furthermore, the serving layer introduces advanced post-processing: isolating the facial region via convex hull and compositing it seamlessly into Subject A's or Subject B's original hair/background using Poisson blending (`cv2.seamlessClone`).

src/app.py - Caching optimization & Poisson seamless cloning for production serving:

```
@st.cache_data(show_spinner=False)
def prepare_pair(file_bytes_a, file_bytes_b, target_margin=TARGET_MARGIN, size=SQUARE_SIZE):
    # Cached face scale alignment and MediaPipe keypoint detection for low-latency serving
    return pts_a, up_a, pts_b, up_b

def render_morph(up_a, pts_a, up_b, pts_b, alpha, frame_from, triangles=None):
    # Advanced production post-processing: Poisson seamless cloning of blended face into photo frame
    hull = cv2.convexHull(pts_m[:478].astype(np.int32))
    cv2.fillConvexPoly(mask, hull, 255)
    try:
        out = cv2.seamlessClone(blend8, frame8, mask, (x+bw//2, y+bh//2), cv2.NORMAL_CLONE)
    except cv2.error:
        # Fallback for tightly-cropped edge boundary cases: feathered Gaussian blending
        soft = cv2.GaussianBlur(cv2.erode(mask, np.ones((15, 15), np.uint8)), (31, 31), 0)
        out = np.clip(blend * (soft/255.0) + frame * (1 - soft/255.0), 0, 255).astype(np.uint8)
    return out, triangles
```

Summary of ML SDLC Mapping Across Repository Modules

- `src/fetch_data.py` -> Phase 1: Data Sourcing, Filtering (different person split), and BGR Normalization.
- `src/landmarks.py` -> Phase 2: Feature Engineering (3x upscaling, 478-pt MediaPipe mesh, 8 boundary anchors).
- `src/morph.py` -> Phase 3: Core Algorithmic Engine (Delaunay triangulation, piecewise-affine warp, cross-dissolve).
- `src/generate_morphs.py` & `build_showcase.py` -> Phase 4: Batch Verification, Exception Handling, QA Audit Logging.
- `src/app.py` -> Phase 5: Production Deployment (Streamlit UI, caching layer, Poisson seamless cloning post-processing).